Part 1: Introduction to Java

1. **What is Java? Explain its significance in modern software development.**
    - Java is a high-level, object-oriented programming language.
    - It follows the "Write Once, Run Anywhere" (WORA) principle.
    - It is widely used for web, mobile, enterprise, and cloud-based applications.
2. **List and explain the key features of Java.**
    - Platform Independent: Runs on any system with a Java Virtual Machine (JVM).
    - Object-Oriented: Follows OOP principles like inheritance, polymorphism, and encapsulation.
    - Secure and Robust: Includes built-in security features and strong memory management.
3. **What is the difference between compiled and interpreted languages? Where does Java fit in?**
    - Compiled Languages (e.g., C, C++): Convert source code into machine code before execution.
    - Interpreted Languages (e.g., Python, JavaScript): Convert code line-by-line at runtime.
    - Java: Uses a hybrid approach—compiles to bytecode (via JVM) and interprets it at runtime.
4. **Explain the concept of platform independence in Java.**
    - Java code is compiled into bytecode, not machine-specific code.
    - The JVM interprets bytecode on any OS, making Java platform-independent.
    - This enables seamless execution across different environments without modification.
5. **What are the various applications of Java in the real world?**
    - Web & Mobile Development: Used in Android apps, enterprise applications, and web frameworks.
    - Big Data & Cloud Computing: Powers Hadoop, Spark, and cloud services.
    - Embedded Systems & IoT: Runs on smart devices, sensors, and automotive software.

Part 2: History of Java

1. **Who developed Java and when was it introduced?**
   - Java was developed by James Gosling and his team at Sun Microsystems.
   - It was introduced in 1995 as part of the Java Development Kit (JDK) 1.0.
   - Oracle Corporation later acquired Sun Microsystems in 2010 and now maintains Java.
2. **What was Java initially called? Why was its name changed?**
   - Java was initially called Oak after an oak tree outside James Gosling's office.
   - The name was changed to Java because "Oak" was already trademarked.
   - "Java" was inspired by Java coffee, reflecting the language's speed and energy.
3. **Describe the evolution of Java versions from its inception to the present.**
   - Java 1.0 (1995): First stable release with core features like OOP and platform independence.
   - Java 5 (2004): Introduced generics, enhanced for-loops, and annotations.
   - Java 8 (2014): Major update with Lambda expressions, Streams, and functional programming.
   - Java 17 (2021, LTS): Improved performance, sealed classes, and pattern matching.
   - Latest versions (Java 21 in 2023): Virtual threads, structured concurrency, and performance enhancements.
4. **What are some of the major improvements introduced in recent Java versions?**
   - Java 8: Lambda expressions, Streams, and functional interfaces.
   - Java 11: Removed deprecated APIs, introduced HTTP Client, and new GC improvements.
   - Java 21: Virtual threads for high-performance concurrency and pattern matching updates.
5. **How does Java compare with other programming languages like C++ and Python in terms of evolution and usability?**
   - Java vs. C++: Java has automatic memory management (Garbage Collection), while C++ requires manual memory handling.
   - Java vs. Python: Java is faster and better for enterprise applications, while Python is easier for beginners and widely used in AI/ML.
   - Evolution: Java consistently adds features for security, scalability, and performance, keeping it relevant across industries.

Part 3: Data Types in Java

1. **Explain the importance of data types in Java.**
   - Define the kind of data a variable can store.
   - Ensure memory efficiency and prevent errors.
   - Enable type safety and improve code reliability.
2. **Differentiate between primitive and non-primitive data types.**
   - Primitive: Predefined by Java (e.g., int, char), stores simple values.
   - Non-Primitive: User-defined or complex types (e.g., String, Array, Class).
   - Non-primitive types require more memory and can have methods**.**
3. **List and briefly describe the eight primitive data types in Java.**
   - byte (1 byte): Stores integers from -128 to 127.
   - short (2 bytes): Stores integers from -32,768 to 32,767.
   - int (4 bytes): Stores integers from -2^{31} to $2^{31}-1$.
   - long (8 bytes): Stores large integers from -2^{63} to $2^{63}-1$.
   - float (4 bytes): Stores decimal numbers (single precision).
   - double (8 bytes): Stores decimal numbers (double precision).
   - char (2 bytes): Stores a single character (Unicode).
   - boolean (1 bit): Stores true or false.
4. **Provide examples of how to declare and initialize different data types.**

```
int num = 10;
double price = 99.99;
char letter = 'A';
boolean isActive = true;
String name = "Java";
```

5. **What is type casting in Java? Explain with an example.**
   - Converting one data type into another.
   - Implicit (Widening): Smaller to larger type (automatic).
     - int a = 10;
     - double b = a; // Implicit: int to double
   - Explicit (Narrowing): Larger to smaller type (manual).

- double x = 99.99;
- int y = (int) x; // Explicit: double to int

6. **Discuss the concept of wrapper classes and their usage in Java.**
   - Convert primitive data types into objects (Integer, Double, etc.).
   - Enable usage of primitive types in collections (e.g., ArrayList<Integer>).
   - Provide utility methods (e.g., Integer.parseInt("123")).
7. **What is the difference between static and dynamic typing? Where does Java stand?**
   - Static Typing: Type is checked at compile-time (e.g., Java, C++).
   - Dynamic Typing: Type is checked at runtime (e.g., Python, JavaScript).
   - Java is statically typed, meaning variables must have a declared type.

Part 4: Java Development Kit (JDK)

1. **What is JDK? How does it differ from JRE and JVM?**
   - JDK (Java Development Kit): Includes tools for developing, compiling, and running Java applications.
   - JRE (Java Runtime Environment): Provides only the runtime libraries and JVM to execute Java programs.
   - JVM (Java Virtual Machine): Interprets and runs Java bytecode on any platform
2. **Explain the main components of JDK.**
   - Compiler (javac): Converts Java source code into bytecode.
   - JVM (java): Runs Java applications using bytecode.
   - Debugger (jdb): Helps debug Java applications.
3. **Describe the steps to install JDK and configure Java on your system.**
   - Download JDK from Oracle or OpenJDK.
   - Install and set PATH and CLASSPATH variables.
   - Verify installation using java -version and javac -version.
4. **Write a simple Java program to print "Hello, World!" and explain its structure.**
   - **class HelloWorld {**
   - **public static void main(String[] args) {**
   - **System.out.println("Hello, World!");**
   - **}**

- **}**
- class HelloWorld → Defines a class.
- public static void main(String[] args) → Entry point of Java programs.
- System.out.println("Hello, World!"); → Prints output to the console.

5. **What is the significance of the PATH and CLASSPATH environment variables in Java?**
   - PATH: Helps locate Java commands (javac, java) from any terminal location.
   - CLASSPATH: Defines locations of Java classes and libraries needed at runtime.
6. **What are the differences between OpenJDK and Oracle JDK?**
   - OpenJDK: Open-source, free, and community-maintained.
   - Oracle JDK: Licensed, with additional performance and support features.
   - Both follow the same Java specifications and functionality.
7. **Explain how Java programs are compiled and executed.**
   - Compilation: javac MyProgram.java → Converts Java code into bytecode (.class).
   - Execution: java MyProgram → JVM loads and runs the bytecode.
8. **What is Just-In-Time (JIT) compilation, and how does it improve Java performance?**
   - Converts frequently used bytecode into native machine code at runtime.
   - Reduces execution time by optimizing code on-the-fly.
   - Balances interpretation and direct execution for better performance.
9. **Discuss the role of the Java Virtual Machine (JVM) in program execution.**
   - Loads, verifies, and executes Java bytecode.
   - Provides memory management (Garbage Collection).
   - Enables platform independence by running bytecode on any OS with a JVM.